Hakari API

Evidências de funcionamento
SOA e Web Services · C# Software Development
Global Solution 2026/2 · CitizenTech · FIAP

Aluno: Lucas Yuki — RM 554865

O Hakari é iluminação pública que se reporta sozinha: um nó ESP32 em cada poste percebe que a lâmpada apagou e avisa por MQTT. Esta API em C#/.NET 10 é o centro da solução: transforma os alertas dos nós em ocorrências, atende o app do cidadão (consultar postes, reportar, apoiar) e o da manutenção (fila por severidade, assumir, resolver), e guarda tudo num PostgreSQL.

Arquitetura: solução em 4 projetos. Domain (entidades e regras), Infrastructure (EF Core, PostgreSQL, BCrypt), Api (controllers REST, JWT, Swagger, workers MQTT) e Tests (xUnit). As dependências apontam para dentro: Api → Infrastructure → Domain.

Como rodar: na pasta api/, docker compose up -d (sobe o PostgreSQL) e dotnet run --project src/Hakari.Api. O Swagger abre em http://localhost:5080/swagger. Detalhes no api/README.md.

Todos os dados das telas são fictícios (e-mails @exemplo.com e credenciais de teste).

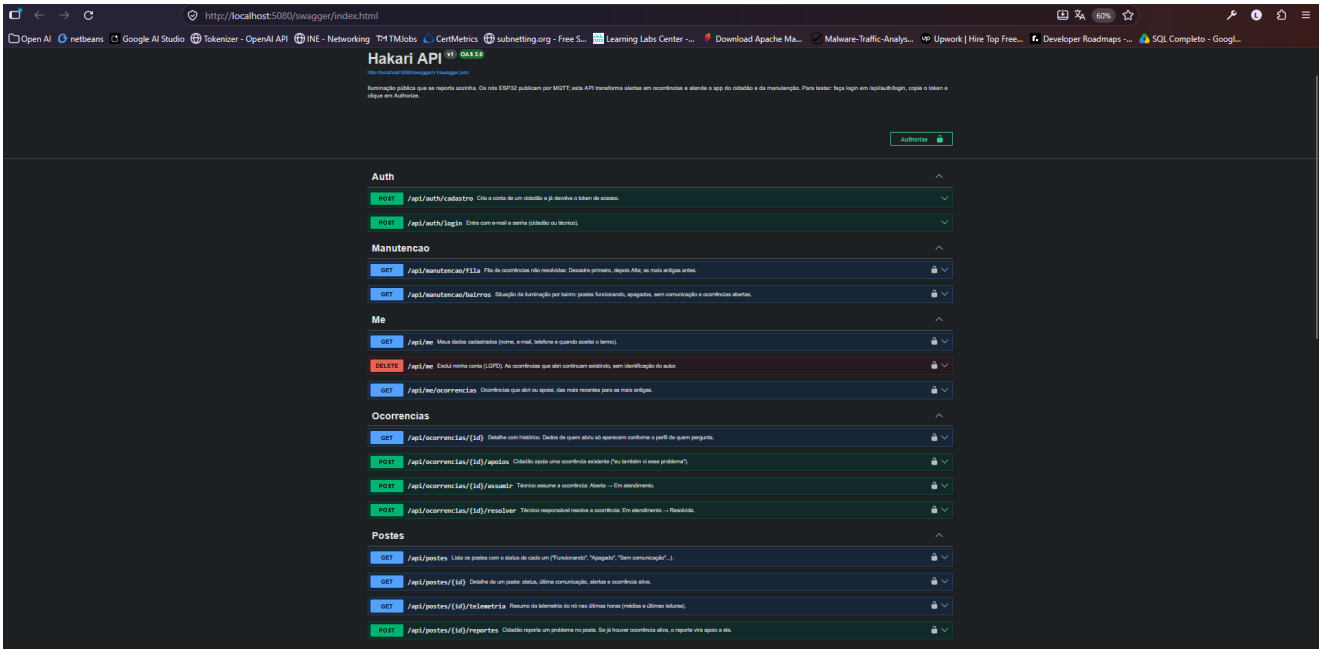
Resumo das evidências

Evidência	O que comprova
1. Documentação automática (Swagger / OpenAPI)	SOA: documentação de serviço · C#: Web API com controllers
2. Cadastro de cidadão: 201 Created	SOA: verbo e status HTTP corretos · C#: autenticação JWT
3. Mesmo e-mail de novo: 409 Conflict	SOA: tratamento de erros padronizado · C#: exceções de domínio
4. Lista de postes com status, ao lado do nó no Wokwi	SOA: recurso REST protegido · integração com o IoT
5. Cidadão reporta um problema: 201 Created	SOA: criação de recurso com Location · C#: regra de negócio no domínio
6. Fila do técnico ordenada por severidade	SOA: autorização por perfil (role) · C#: LINQ / EF Core
7. Regra de transição: resolver sem assumir é 422	C#: encapsulamento, o status só muda pelos métodos da entidade
8. Técnico assume a ocorrência: histórico completo	C#: entidades com histórico · regra de privacidade do autor
9. Integração MQTT: alertas do Wokwi chegando na API	SOA: comunicação assíncrona por mensageria · C#: BackgroundService
10. Banco PostgreSQL criado pelas migrações	C#: EF Core code-first com migrações · persistência relacional
11. Testes automatizados	C#: testes unitários · qualidade

1. Documentação automática (Swagger / OpenAPI)

Os 15 endpoints REST aparecem agrupados por recurso: Auth, Manutencao, Me, Ocorrencias e Postes. O cadeado marca só as rotas que exigem token JWT; cadastro e login ficam livres.

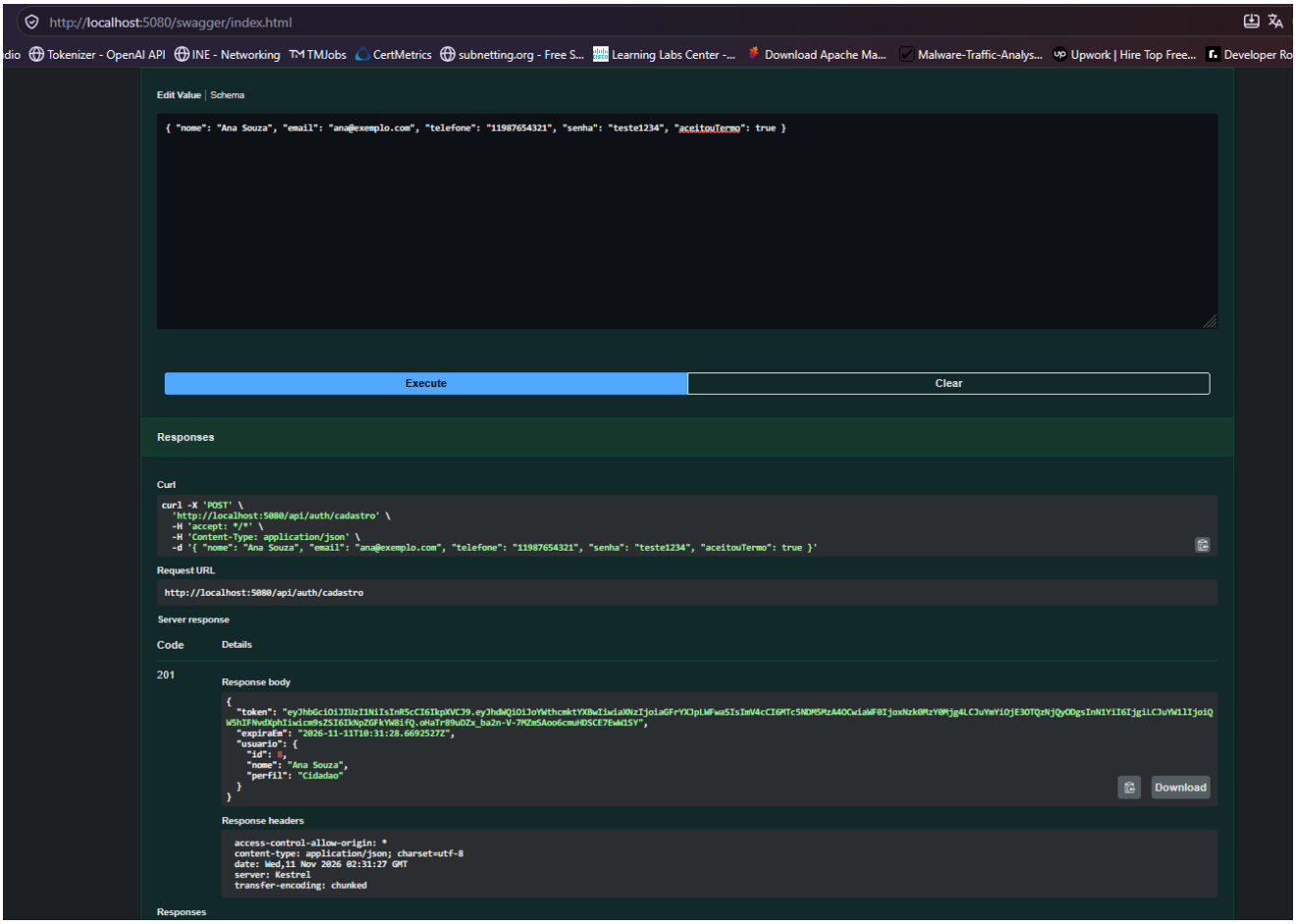
Comprova: SOA: documentação de serviço · C#: Web API com controllers



2. Cadastro de cidadão: 201 Created

POST /api/auth/cadastro cria a conta (dados fictícios) e já devolve o token JWT, a expiração e o perfil "Cidadao". A senha é guardada só como hash BCrypt.

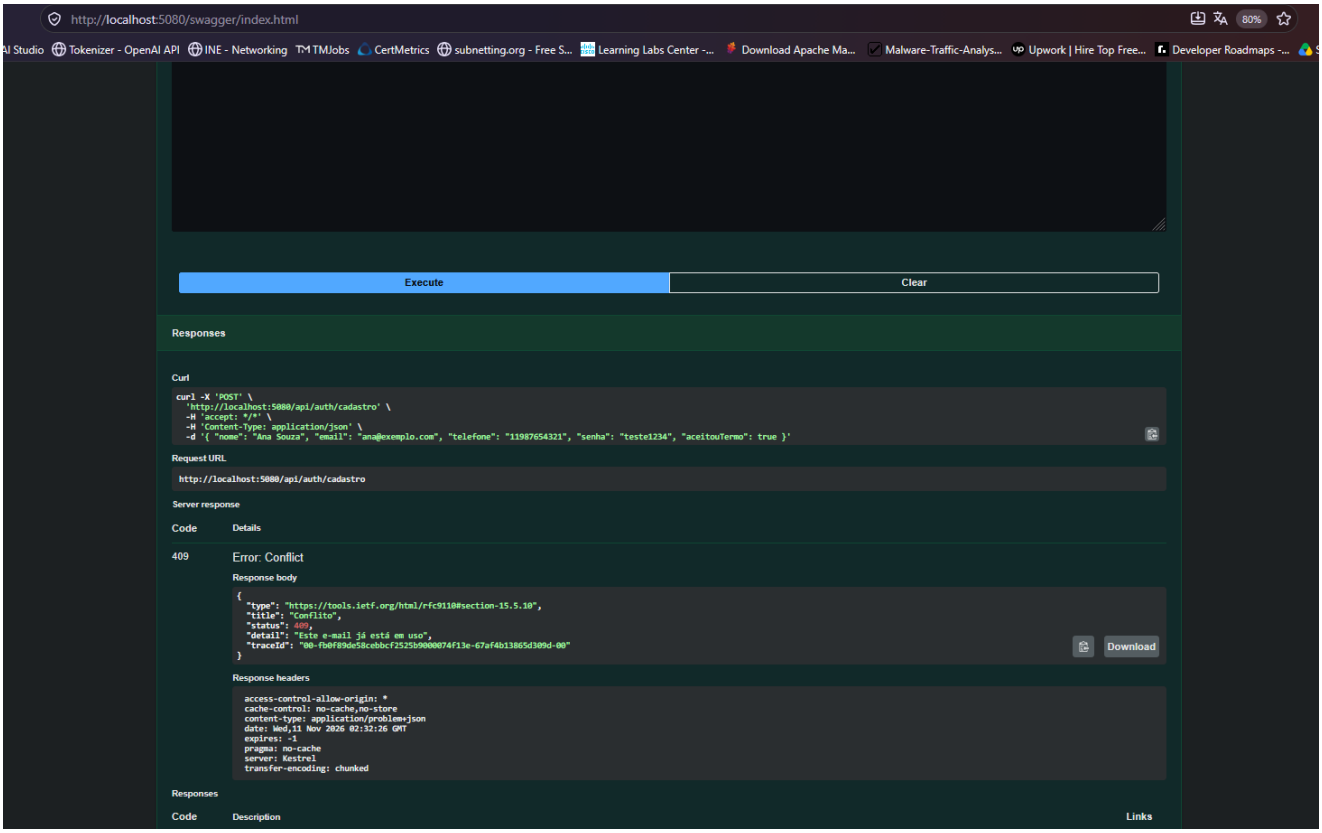
Comprova: SOA: verbo e status HTTP corretos · C#: autenticação JWT



3. Mesmo e-mail de novo: 409 Conflict

A regra de e-mail único devolve 409 no formato padrão ProblemDetails (RFC 9110), com a mensagem "Este e-mail já está em uso". Os erros de domínio viram respostas HTTP num tratador central.

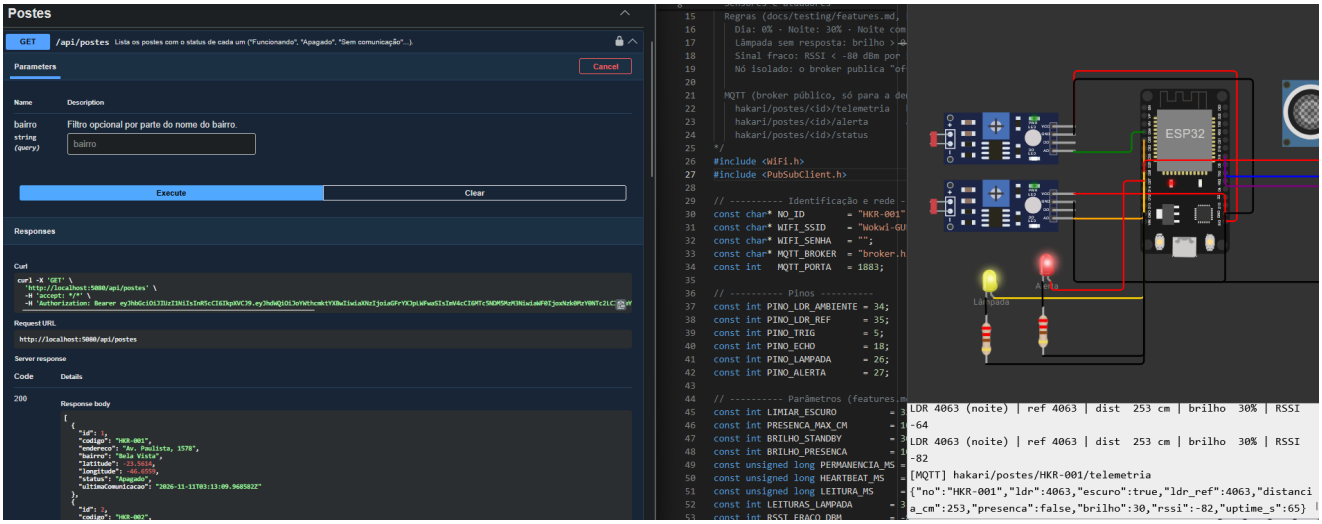
Comprova: SOA: tratamento de erros padronizado · C#: exceções de domínio



4. Lista de postes com status, ao lado do nó no Wokwi

GET /api/postes, autenticado, devolve os postes com status calculado no domínio ("Apagado", "Sem comunicação", "Aguardando instalação"). À direita, o ESP32 no Wokwi publicando a telemetria do HKR-001 no tópico hakari/postes/HKR-001/telemetria.

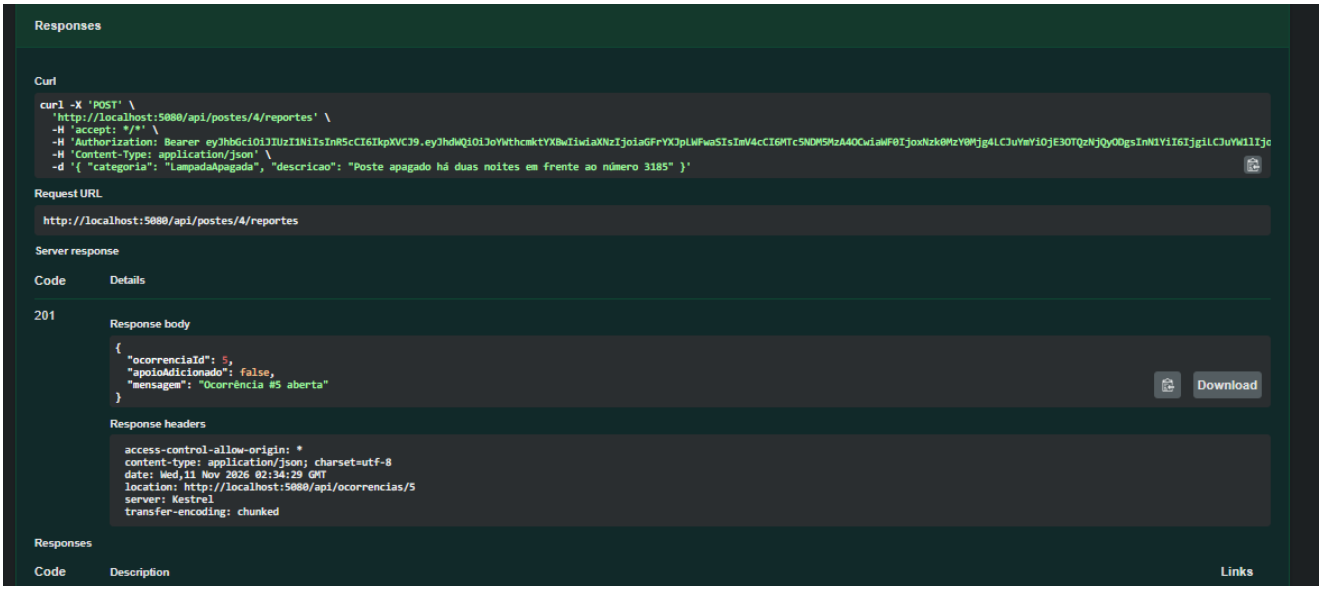
Comprova: SOA: recurso REST protegido · integração com o IoT



5. Cidadão reporta um problema: 201 Created

POST /api/postes/4/reportes abre a ocorrência #5 e devolve o cabeçalho Location apontando para /api/ocorrencias/5. Se já existisse ocorrência ativa no poste, o reporte viraria apoio (200).

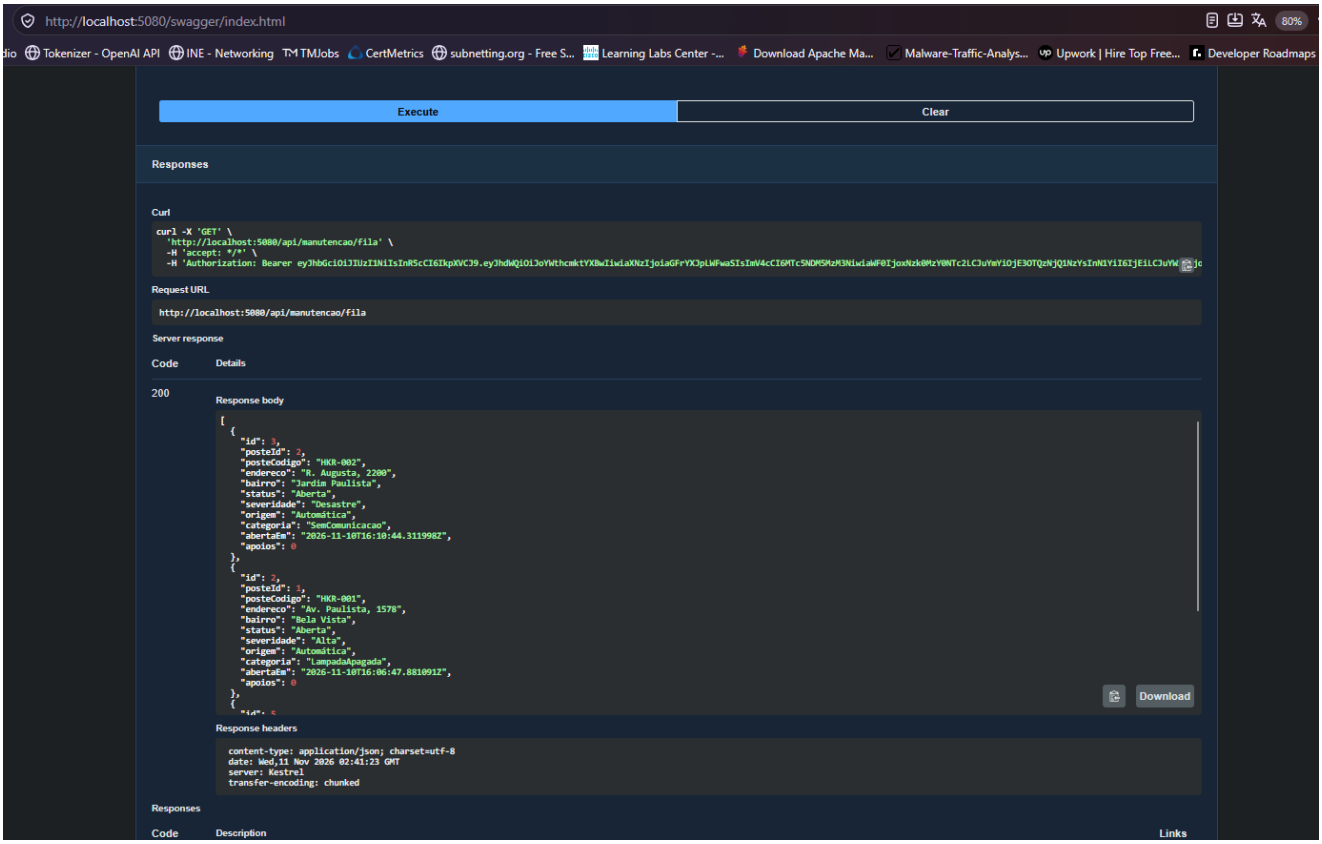
Comprova: SOA: criação de recurso com Location · C#: regra de negócio no domínio



6. Fila do técnico ordenada por severidade

GET /api/manutencao/fila, só para o perfil Técnico, lista as ocorrências não resolvidas: Desastre primeiro (HKR-002, nó isolado), depois Alta, e as mais antigas antes.

Comprova: SOA: autorização por perfil (role) · C#: LINQ / EF Core



7. Regra de transição: resolver sem assumir é 422

A resposta documentada no Swagger para POST /api/ocorrencias/{id}/resolver: o domínio recusa resolver uma ocorrência que ainda não foi assumida, com "Assuma a ocorrência antes de resolvê-la". A mesma regra é coberta pelos testes automatizados (evidência 11).

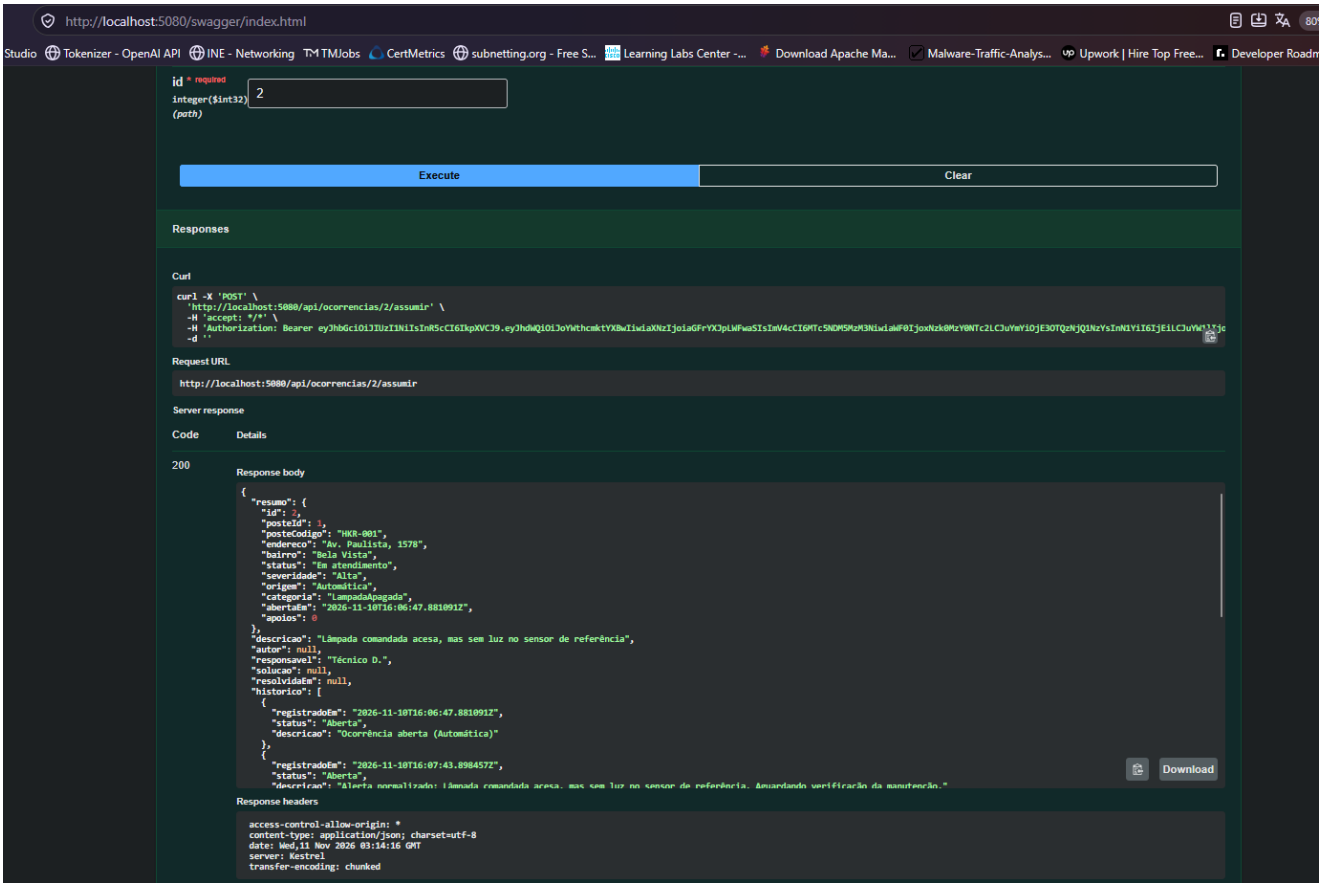
Comprova: C#: encapsulamento, o status só muda pelos métodos da entidade



8. Técnico assume a ocorrência: histórico completo

POST /api/ocorrencias/2/assumir muda o status para "Em atendimento" e registra o responsável (nome abreviado, "Técnico D."). O histórico guarda cada evento: a abertura automática pelo nó e a normalização do alerta.

Comprova: C#: entidades com histórico · regra de privacidade do autor



9. Integração MQTT: alertas do Wokwi chegando na API

Log da API: o MqttWorker recebe os alertas do HKR-001 (Aviso de sinal fraco, Alta de lâmpada sem resposta), o status offline do last will e, depois de 3 minutos sem mensagens, o IsolamentoWorker marca o nó como isolado.

Comprova: SOA: comunicação assíncrona por mensageria · C#: BackgroundService

```
PS C:\Users\Predator\Desktop\HAKARI_GS\api> dotnet run --project src/Hakari.Api
info: Hakari.Api.Aplicacao.MonitorService[0]
      Alerta Aviso SinalFraco aberto no nó HKR-001
info: Hakari.Api.Aplicacao.MonitorService[0]
      Alerta Alta LampadaSemResposta aberto no nó HKR-001
warn: Hakari.Api.Aplicacao.MonitorService[0]
      Nó HKR-001 informou status offline
warn: Hakari.Api.Aplicacao.MonitorService[0]
      Nó HKR-001 isolado: sem mensagens desde 2026-11-11 03:14:06Z
warn: Hakari.Api.Workers.IsolamentoWorker[0]
      1 nó(s) marcado(s) como isolado(s)
info: Microsoft.Hosting.Lifetime[0]
      Application is shutting down...
```

10. Banco PostgreSQL criado pelas migrações

As 7 tabelas do domínio e a __EFMigrationsHistory, criadas pela migração Inicial do EF Core (docker exec hakari-db psql -U hakari -d hakari -c "\dt").

Comprova: C#: EF Core code-first com migrações · persistência relacional

```
                List of relations
 Schema |          Name          | Type  | Owner
--------+------------------------+-------+--------
 public | __EFMigrationsHistory  | table | hakari
 public | alertas                | table | hakari
 public | apoios                 | table | hakari
 public | historico_ocorrencias  | table | hakari
 public | leituras               | table | hakari
 public | ocorrencias            | table | hakari
 public | postes                 | table | hakari
 public | usuarios               | table | hakari
(8 rows)
```

11. Testes automatizados

dotnet test: 41 testes xUnit aprovados, 0 falhas. Eles cobrem as regras do domínio e os casos de uso, com banco em memória e relógio falso.

Comprova: C#: testes unitários · qualidade

```
PS C:\Users\Predator\Desktop\HAKARI_GS\api> dotnet test
Restauração concluída (1,2s)
  Hakari.Domain net10.0 êxito (0,3s) → src\Hakari.Domain\bin\Debug\net10.0\Hakari.Domain.dll
  Hakari.Infrastructure net10.0 êxito (0,3s) → src\Hakari.Infrastructure\bin\Debug\net10.0\Hakari.Infrastructure.dll
  Hakari.Api net10.0 êxito (0,6s) → src\Hakari.Api\bin\Debug\net10.0\Hakari.Api.dll
  Hakari.Tests net10.0 êxito (0,6s) → tests\Hakari.Tests\bin\Debug\net10.0\Hakari.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.65]   Discovering: Hakari.Tests
[xUnit.net 00:00:00.70]   Discovered:  Hakari.Tests
[xUnit.net 00:00:00.73]   Starting:    Hakari.Tests
[xUnit.net 00:00:01.94]   Finished:    Hakari.Tests
  Hakari.Tests teste net10.0 êxito (3,9s)

Resumo do teste: total: 41; falhou: 0; bem-sucedido: 41; ignorado: 0; duração: 3,9s
Construir êxito em 7,1s
PS C:\Users\Predator\Desktop\HAKARI_GS\api>
```